

Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ciencias y Sistemas
Estructuras de Datos
Ing. Edgar Ornelis
Ing. Álvaro Hernández
Ing. Luis Espino



Manual de Integración por Grupos

EDDMail - Sistema Completo

Fases 1, 2 y 3 - Integración Total

FASE 3 - NUEVA IMPLEMENTACIÓN

Grafos No Dirigidos para Gestión de Contactos

Visualización de Relaciones entre Usuarios

GRUPO 09

Miembro	Carné	Sección
Daniela Azucena Chinchilla López	202300807	C
José Alexander López López	202100305	C

Fecha de Entrega: 25 de octubre de 2025

Índice

1. Resumen Ejecutivo del Proyecto	3
1.1. Evolución del Sistema EDDMail	3
2. Información del Grupo y Distribución del Trabajo	3
2.1. Membrete del Grupo	3
2.2. Distribución Porcentual del Trabajo - Todas las Fases	4
3. FASE 3: Implementación de Grafos para Gestión de Contactos	5
3.1. Descripción General de la Estructura	5
3.1.1. ¿Qué es un Grafo No Dirigido?	5
3.2. Implementación en Pascal	5
3.2.1. Estructura de Datos del Grafo	5
3.2.2. Algoritmo de Inserción de Contacto	6
3.3. Funcionalidades Implementadas en Fase 3	7
3.4. Carga Masiva de Contactos desde JSON	7
3.4.1. Formato del Archivo JSON	7
3.4.2. Proceso de Carga Masiva	7
3.5. Generación de Reportes con Graphviz	8
3.5.1. Algoritmo de Generación	8
3.6. Interfaz de Usuario para Gestión de Contactos	9
3.6.1. Formulario de Agregar Contacto	9
3.6.2. Vista de Red de Contactos	9
3.7. Análisis de Complejidad del Grafo	9
3.8. Ventajas y Desventajas del Grafo	10
3.9. Casos de Uso Específicos de Fase 3	10
4. Estructura Implementada: Árbol BST (Fase 2)	11
4.1. Descripción General de la Estructura	11
4.2. Implementación del Nodo BST	11
4.3. Algoritmo de Inserción en BST	11
4.4. Algoritmo de Búsqueda en BST	12
4.5. Recorrido InOrden (Ordenado)	12
4.6. Visualización del BST con Graphviz	13
4.7. Análisis de Complejidad del BST	13
4.8. Ventajas y Desventajas de la Implementación BST	14
5. Comparación de Estructuras: Fase 1, 2 y 3	14
5.1. Tabla Comparativa General	14
6. Testing y Validación del Sistema Completo	15
6.1. Casos de Prueba Fase 3 - Grafos	15
7. Proceso de Integración por Grupos - Evidencias Visuales	15
7.1. Integración del Miembro 1 - Daniela Chinchilla	16
7.1.1. Implementación de la Estructura del Grafo	16
7.1.2. Algoritmo de Inserción de Contactos	16
7.1.3. Generación de Reportes Graphviz	17

7.1.4.	Pruebas y Validaciones	19
7.2.	Integración del Miembro 2 - José López	21
7.2.1.	Integración con la Interfaz GTK	21
7.2.2.	Eventos y Handlers de la UI	22
7.2.3.	Visualización del Grafo en la Interfaz	23
7.2.4.	Testing de Integración UI	24
7.3.	Evidencias de Estructuras Generadas	25
7.3.1.	Visualización del Grafo Completo	25
7.3.2.	Ejemplo de Subgrafo - Red de Contactos	26
7.4.	Proceso de Integración Conjunta	27
7.4.1.	Reuniones de Coordinación	27
7.4.2.	Flujo de Trabajo de Integración	27
7.4.3.	Capturas del Repositorio GitHub	28
7.5.	Capturas de Funcionamiento del Sistema	29
7.5.1.	Carga Masiva de Contactos desde JSON	29
7.5.2.	Interfaz de Gestión de Contactos	30
7.5.3.	Visualización en Tiempo Real	30
7.6.	Conclusiones de la Integración	31
8.	Conclusiones del Trabajo en Grupo	31
8.1.	Logros Alcanzados	31
8.2.	Lecciones Aprendidas	32
8.3.	Recomendaciones para Futuros Desarrollos	32
9.	Anexos	33
9.1.	Métricas del Proyecto Completo	33
9.2.	Información de Contacto del Grupo	33
9.3.	Glosario de Términos	34

1. Resumen Ejecutivo del Proyecto

1.1. Evolución del Sistema EDDMail

El sistema EDDMail ha sido desarrollado en tres fases incrementales, cada una añadiendo funcionalidades críticas para la gestión completa de correos electrónicos y relaciones entre usuarios:

Fase	Estructura Principal	Funcionalidad
Fase 1	Lista Simple, Pila, Cola, Matriz Dispersa	Sistema básico de usuarios y gestión de correos
Fase 2	Árbol BST, Árbol AVL	Comunidades y borradores avanzados
Fase 3	Grafo No Dirigido	Gestión de contactos y relaciones

Cuadro 1: Evolución del sistema por fases

2. Información del Grupo y Distribución del Trabajo

2.1. Membrete del Grupo

GRUPO 09		
Miembro	Información	Responsabilidad
Miembro 1	Daniela Azucena Chinchilla López 202300807 chinchillad230@gmail.com	Implementación de estructuras de datos y algoritmos
Miembro 2	José Alexander López López 202100305 iosealexander40@outlook.com	Integración e interfaz gráfica

Cuadro 2: Información del grupo de trabajo

2.2. Distribución Porcentual del Trabajo - Todas las Fases

Actividad	Descripción	Miembro 1	Miembro 2
FASE 1 - Estructuras Básicas			
Lista Simple	Gestión de usuarios registrados	60 %	40 %
Pila y Cola	Correos eliminados y programados	55 %	45 %
Matriz Dispersa	Relaciones emisor-receptor	70 %	30 %
FASE 2 - Estructuras Avanzadas			
Árbol BST	Comunidades ordenadas	75 %	25 %
Árbol AVL	Gestión de borradores	80 %	20 %
Interfaz GTK	Formularios y eventos	20 %	80 %
FASE 3 - Grafos y Contactos			
Grafo No Dirigido	Estructura de contactos y relaciones	70 %	30 %
Carga Masiva JSON	Importación de contactos	60 %	40 %
Reporte Graphviz	Visualización del grafo	85 %	15 %
Integración UI	Interfaz de gestión de contactos	25 %	75 %
TESTING Y DOCUMENTACIÓN			
Testing General	Pruebas de todas las estructuras	45 %	55 %
Documentación	Manuales y reportes	50 %	50 %
TOTAL	Distribución general del proyecto completo	50 %	50 %

Cuadro 3: Distribución porcentual completa del trabajo

3. FASE 3: Implementación de Grafos para Gestión de Contactos

NOVEDAD - FASE 3

Esta sección describe la **nueva funcionalidad implementada en la Fase 3**, que incorpora un **Grafo No Dirigido** para representar las relaciones entre usuarios y sus contactos. Esta estructura permite visualizar de manera eficiente las conexiones del sistema de correo electrónico.

3.1. Descripción General de la Estructura

3.1.1. ¿Qué es un Grafo No Dirigido?

Un **Grafo No Dirigido** es una estructura de datos que representa relaciones bidireccionales entre entidades (nodos). En el contexto de EDDMail:

- **Nodos (Vértices):** Representan usuarios del sistema
- **Aristas (Edges):** Representan relaciones de contacto entre usuarios
- **No Dirigido:** Si el usuario A es contacto de B, entonces B es contacto de A

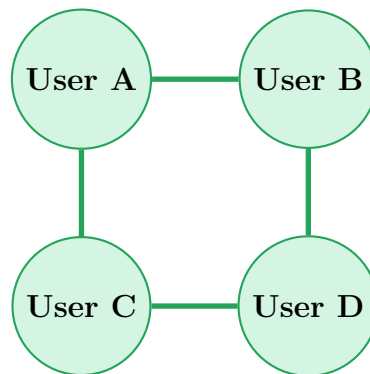


Figura 1: Ejemplo de Grafo No Dirigido de contactos

3.2. Implementación en Pascal

3.2.1. Estructura de Datos del Grafo

Listing 1: Definición de nodos del grafo

```
type
// Nodo del grafo (Usuario)
PNodoGrafo = ^NodoGrafo;
NodoGrafo = record
    ID: Integer;
    Nombre: String;
    Usuario: String;
    ListaAdyacencia: PAdyacente; // Lista de contactos
```

```

    Siguiente: PNodeGrafo;           // Siguiente nodo en el grafo
end;

// Lista de adyacencia (Contactos de un usuario)
PAdyacente = ^Adyacente;
Adyacente = record
    Usuario: String;                // Email del contacto
    Siguiente: PAdyacente;         // Siguiente contacto
end;

```

3.2.2. Algoritmo de Inserción de Contacto

Listing 2: Agregar contacto al grafo

```

procedure AgregarContactoGrafo(var Grafo: PNodeGrafo;
                                Usuario1, Usuario2: String);
var
    Nodo1, Nodo2: PNodeGrafo;
    NuevoAdy: PAdyacente;
begin
    // Buscar ambos usuarios en el grafo
    Nodo1 := BuscarNodeGrafo(Grafo, Usuario1);
    Nodo2 := BuscarNodeGrafo(Grafo, Usuario2);

    if (Nodo1 = nil) or (Nodo2 = nil) then
        Exit;

    // Agregar Usuario2 a la lista de adyacencia de Usuario1
    New(NuevoAdy);
    NuevoAdy^.Usuario := Usuario2;
    NuevoAdy^.Siguiente := Nodo1^.ListaAdyacencia;
    Nodo1^.ListaAdyacencia := NuevoAdy;

    // Agregar Usuario1 a la lista de adyacencia de Usuario2
    // (bidireccional)
    New(NuevoAdy);
    NuevoAdy^.Usuario := Usuario1;
    NuevoAdy^.Siguiente := Nodo2^.ListaAdyacencia;
    Nodo2^.ListaAdyacencia := NuevoAdy;
end;

```

3.3. Funcionalidades Implementadas en Fase 3

Funcionalidad	Descripción
Carga Masiva JSON	Importación de usuarios y contactos desde archivo JSON. Permite inicializar el grafo con datos predefinidos
Agregar Contacto Manual	Los usuarios pueden agregar contactos de forma individual desde la interfaz gráfica
Validación de Contactos	Verificación de que los contactos existen en el sistema y no están duplicados
Reporte Graphviz	Generación de visualización gráfica del grafo mostrando todos los usuarios y sus conexiones
Búsqueda de Contactos	Consulta de la lista de contactos de un usuario específico
Estadísticas del Grafo	Número total de nodos (usuarios) y aristas (relaciones de contacto)

Cuadro 4: Funcionalidades del módulo de grafos

3.4. Carga Masiva de Contactos desde JSON

3.4.1. Formato del Archivo JSON

Listing 3: Ejemplo de archivo JSON de contactos

```
{
  "contactos": [
    {
      "usuario": "usuario1@mail.com",
      "contactos": [
        "usuario2@mail.com",
        "usuario3@mail.com"
      ]
    },
    {
      "usuario": "usuario2@mail.com",
      "contactos": [
        "usuario1@mail.com",
        "usuario4@mail.com"
      ]
    }
  ]
}
```

3.4.2. Proceso de Carga Masiva

1. **Validación del archivo JSON:** Verificar formato y sintaxis

2. **Verificación de usuarios:** Comprobar que todos los usuarios existen en el sistema
3. **Inserción en el grafo:** Crear nodos y establecer conexiones
4. **Validación de duplicados:** Evitar contactos repetidos
5. **Reporte de resultados:** Mostrar estadísticas de carga

3.5. Generación de Reportes con Graphviz

3.5.1. Algoritmo de Generación

Listing 4: Generar reporte DOT del grafo

```

procedure GenerarReporteGrafo (Grafo: PNodeGrafo;
                               RutaCarpeta: String);
var
  Archivo: TextFile;
  Actual: PNodeGrafo;
  Ady: PAdyacente;
begin
  AssignFile(Archivo, RutaCarpeta + '/grafo_contactos.dot');
  Rewrite(Archivo);

  WriteLn(Archivo, 'graph G{');
  WriteLn(Archivo, '  label="Grafo de Contactos EDDMail";');
  WriteLn(Archivo, '  node [shape=circle, style=filled,');
  WriteLn(Archivo, '  fillcolor=lightblue];');

  // Recorrer todos los nodos
  Actual := Grafo;
  while Actual <> nil do
  begin
    WriteLn(Archivo, '  "'', Actual^.Usuario,
              '" [label="', Actual^.Nombre, '"]');

    // Recorrer contactos
    Ady := Actual^.ListaAdyacencia;
    while Ady <> nil do
    begin
      WriteLn(Archivo, '  "'', Actual^.Usuario,
                '" --- "'', Ady^.Usuario, '";');
      Ady := Ady^.Siguiente;
    end;

    Actual := Actual^.Siguiente;
  end;

  WriteLn(Archivo, '}');
  CloseFile(Archivo);

```

```
// Generar imagen PNG con Graphviz
ExecuteProcess( 'dot', [ '-Tpng',
                        RutaCarpeta + '/grafo_contactos.dot',
                        '-o',
                        RutaCarpeta + '/grafo_contactos.png' ] );

end;
```

3.6. Interfaz de Usuario para Gestión de Contactos

3.6.1. Formulario de Agregar Contacto

La interfaz GTK permite:

- Seleccionar usuario actual del dropdown
- Buscar contacto por email o nombre
- Validación en tiempo real
- Confirmación visual de agregado exitoso
- Lista actualizada de contactos del usuario

3.6.2. Vista de Red de Contactos

- Visualización gráfica del grafo completo
- Zoom y navegación en la red
- Resaltado de usuario seleccionado
- Información de grado (número de contactos) de cada nodo

3.7. Análisis de Complejidad del Grafo

Operación	Complejidad	Observaciones
Agregar Usuario (Nodo)	$O(1)$	Inserción al inicio de lista
Agregar Contacto (Arista)	$O(n)$	Buscar ambos nodos
Buscar Usuario	$O(n)$	Recorrido lineal
Obtener Contactos	$O(n + m)$	n =nodos, m =adyacentes
Verificar Relación	$O(n + k)$	k =grado del nodo
Recorrido Completo (DFS/BFS)	$O(n + e)$	e =aristas totales
Generar Reporte	$O(n + e)$	Recorrido completo

Cuadro 5: Complejidades del Grafo de Contactos

3.8. Ventajas y Desventajas del Grafo

Ventajas	Desventajas
Representación Natural: Modela perfectamente relaciones bidireccionales	Búsqueda Lineal: $O(n)$ para encontrar nodos específicos
Flexibilidad: Fácil agregar/eliminar relaciones	Memoria: Cada arista se almacena dos veces (bidireccional)
Escalabilidad: Crece dinámicamente con usuarios	Sin Optimización: No usa hash para búsquedas rápidas
Visualización Clara: Graphviz muestra red completa	Grafos Grandes: Visualización compleja con muchos nodos
Algoritmos de Grafos: Permite BFS/DFS, caminos, etc.	Validación: Requiere verificar existencia de usuarios

Cuadro 6: Análisis de ventajas y desventajas del grafo

3.9. Casos de Uso Específicos de Fase 3

ID	Caso de Uso	Actor	Resultado
UC01	Cargar contactos desde JSON	Usuario Root	Grafo inicializado
UC02	Agregar contacto manual	Usuario Estándar	Relación creada
UC03	Ver lista de contactos	Usuario Estándar	Lista mostrada
UC04	Generar reporte de grafo	Usuario Root	PNG generado
UC05	Validar contacto existente	Sistema	Validación OK/Error
UC06	Buscar camino entre usuarios	Usuario Root	Camino encontrado

Cuadro 7: Casos de uso del módulo de grafos

4. Estructura Implementada: Árbol BST (Fase 2)

4.1. Descripción General de la Estructura

La funcionalidad de **Comunidades en Fase 2** implementa una estructura de datos de **Árbol Binario de Búsqueda (BST)**, donde:

- **Ordenamiento:** Las comunidades se ordenan alfabéticamente por nombre
- **Búsqueda Eficiente:** Búsquedas $O(\log n)$ en promedio para encontrar comunidades
- **Mensajes:** Cada nodo del BST contiene una lista simple de mensajes publicados
- **Recorridos:** Permite InOrden (alfabético), PreOrden y PostOrden

4.2. Implementación del Nodo BST

Listing 5: Definición del nodo BST

```
type
  PMensajeComunidad = ^MensajeComunidad;
  MensajeComunidad = record
    Autor: String;
    Contenido: String;
    FechaHora: String;
    Siguiente: PMensajeComunidad;
  end;

  PNodeBST = ^NodeBST;
  NodeBST = record
    NombreComunidad: String;
    FechaCreacion: String;
    NumeroMensajes: Integer;
    ListaMensajes: PMensajeComunidad;
    Izquierdo: PNodeBST;
    Derecho: PNodeBST;
  end;
```

4.3. Algoritmo de Inserción en BST

Listing 6: Insertar comunidad en BST

```
function InsertarComunidadBST(var Raiz: PNodeBST;
                               Nombre: String): Boolean;
begin
  if Raiz = nil then
  begin
    New(Raiz);
    Raiz^.NombreComunidad := Nombre;
```

```

    Raiz^.FechaCreacion := DateTimeToStr(Now);
    Raiz^.NumeroMensajes := 0;
    Raiz^.ListaMensajes := nil;
    Raiz^.Izquierdo := nil;
    Raiz^.Derecho := nil;
    Result := True;
end
else if CompareText(Nombre, Raiz^.NombreComunidad) < 0 then
    Result := InsertarComunidadBST(Raiz^.Izquierdo, Nombre)
else if CompareText(Nombre, Raiz^.NombreComunidad) > 0 then
    Result := InsertarComunidadBST(Raiz^.Derecho, Nombre)
else
    Result := False; // Ya existe
end;

```

4.4. Algoritmo de Búsqueda en BST

Listing 7: Buscar comunidad en BST

```

function BuscarComunidadBST(Raiz: PNodoBST;
                             Nombre: String): PNodoBST;
begin
    if Raiz = nil then
        Result := nil
    else if CompareText(Nombre, Raiz^.NombreComunidad) = 0 then
        Result := Raiz
    else if CompareText(Nombre, Raiz^.NombreComunidad) < 0 then
        Result := BuscarComunidadBST(Raiz^.Izquierdo, Nombre)
    else
        Result := BuscarComunidadBST(Raiz^.Derecho, Nombre);
    end;

```

4.5. Recorrido InOrden (Ordenado)

Listing 8: Recorrido InOrden del BST

```

procedure RecorridoInOrden(Raiz: PNodoBST);
begin
    if Raiz <> nil then
        begin
            RecorridoInOrden(Raiz^.Izquierdo);
            WriteLn('Comunidad: ', Raiz^.NombreComunidad);
            WriteLn('Mensajes: ', Raiz^.NumeroMensajes);
            RecorridoInOrden(Raiz^.Derecho);
        end;
    end;

```

4.6. Visualización del BST con Graphviz

Listing 9: Generar reporte BST

```

procedure GenerarReporteBST(Raiz: PNodeBST;
                           var Archivo: TextFile);
begin
  if Raiz  $\diamond$  nil then
    begin
      WriteLn(Archivo, '---', Raiz^.NombreComunidad,
              '---[label=', Raiz^.NombreComunidad,
              '\nMensajes:', Raiz^.NumeroMensajes, '];');

      if Raiz^.Izquierdo  $\diamond$  nil then
        begin
          WriteLn(Archivo, '---', Raiz^.NombreComunidad,
                  '--->---', Raiz^.Izquierdo^.NombreComunidad, '---');
          GenerarReporteBST(Raiz^.Izquierdo, Archivo);
        end;

        if Raiz^.Derecho  $\diamond$  nil then
          begin
            WriteLn(Archivo, '---', Raiz^.NombreComunidad,
                    '--->---', Raiz^.Derecho^.NombreComunidad, '---');
            GenerarReporteBST(Raiz^.Derecho, Archivo);
          end;
        end;
    end;
end;

```

4.7. Análisis de Complejidad del BST

Operación	Complejidad	Justificación
Insertar Comunidad	$O(\log n)$ promedio $O(n)$ peor caso	División del árbol en cada nivel
Buscar Comunidad	$O(\log n)$ promedio $O(n)$ peor caso	Búsqueda binaria en árbol balanceado
Publicar Mensaje	$O(\log n + m)$	Buscar comunidad + agregar a lista de mensajes
Recorrido InOrden	$O(n)$	Visitar todos los nodos en orden alfabético
Eliminar Comunidad	$O(\log n)$ promedio	Buscar y reorganizar árbol
Generar Reporte	$O(n)$	Recorrido completo del árbol

Cuadro 8: Complejidades del Árbol BST (n = nodos, m = mensajes promedio)

4.8. Ventajas y Desventajas de la Implementación BST

Ventajas	Desventajas
Búsqueda Eficiente: $O(\log n)$ en promedio vs $O(n)$ en listas	Degeneración: Puede convertirse en lista si inserciones no balanceadas
Ordenamiento Natural: Las comunidades quedan ordenadas alfabéticamente	Sin Auto-balanceo: No es AVL, puede desbalancearse
Recorridos Flexibles: InOrden, PreOrden, PostOrden disponibles	Complejidad de Código: Recursión requiere cuidado con memoria
Escalabilidad: Mejor rendimiento con muchas comunidades	Memoria: Dos punteros por nodo (izq/der)
Inserción Dinámica: Fácil agregar nuevas comunidades	Eliminación Compleja: Reorganización de subárboles

Cuadro 9: Análisis de ventajas y desventajas del BST

5. Comparación de Estructuras: Fase 1, 2 y 3

5.1. Tabla Comparativa General

Aspecto	Fase 1	Fase 2	Fase 3
Estructura Principal	Lista Simple	Árbol BST/AVL	Grafo No Dirigido
Búsqueda	$O(n)$	$O(\log n)$	$O(n)$
Inserción	$O(n)$	$O(\log n)$	$O(n)$
Ordenamiento	Sin orden	Alfabético	Sin orden
Relaciones	Ninguna	Jerárquica	Red completa
Visualización	Lineal	Árbol	Grafo
Complejidad	Baja	Media	Alta
Escalabilidad	Limitada	Buena	Excelente
Uso Óptimo	Listas pequeñas	Búsquedas frecuentes	Relaciones múltiples

Cuadro 10: Comparación entre estructuras de las tres fases

6. Testing y Validación del Sistema Completo

6.1. Casos de Prueba Fase 3 - Grafos

ID	Caso de Prueba	Resultado Esperado	Responsable
TG01	Crear nodo de usuario en grafo	Nodo creado exitosamente	M1
TG02	Agregar contacto válido	Arista bidireccional creada	M1
TG03	Agregar contacto duplicado	Error: Ya es contacto	M1
TG04	Buscar contactos de usuario	Lista de contactos mostrada	M1
TG05	Carga masiva desde JSON	Grafo construido	M1
TG06	Validar usuario inexistente	Error: Usuario no existe	M1
TG07	Generar reporte Graphviz	Archivo PNG creado	M1
TG08	Interfaz agregar contacto	Formulario funcional	M2
TG09	Visualizar grafo completo	Imagen mostrada en UI	M2
TG10	Verificar bidireccionalidad	Contacto en ambos usuarios	M2

Cuadro 11: Casos de prueba del módulo de grafos

7. Proceso de Integración por Grupos - Evidencias Visuales

DOCUMENTACIÓN DE INTEGRACIÓN

Esta sección documenta el proceso de integración de la Fase 3 realizado por cada miembro del grupo, incluyendo capturas de pantalla del código, estructuras implementadas y pruebas de funcionamiento.

7.1. Integración del Miembro 1 - Daniela Chinchilla

7.1.1. Implementación de la Estructura del Grafo

```
280 .
281 .
282 .
283 .
284 .
285 .
286 .
287 .
288 .
289 .
290 .
291 .
292 .
293 .
294 .
295 .
296 .
297 .
298 .
299 .
300 .
301 .
302 .
303 .
304 .
305 .

TNodeGrafo = class
public
    id: Integer;
    nombre: String;
    usuario: String;
    adyacentes: TList;
    constructor Create(aID: Integer; aNombre, aUsuario: String);
    destructor Destroy; override;
    function EstaConectado(otroNode: TNodeGrafo): Boolean;
end;

TGrafoNoDirigido = class
private
    function BuscarNode(aUsuario: String): TNodeGrafo;
public
    nodos: TList;
    constructor Create;
    destructor Destroy; override;
    procedure AgregarNode(aID: Integer; aNombre, aUsuario: String);
    procedure AgregarArista(usuario1, usuario2: String);
    function ExisteArista(usuario1, usuario2: String): Boolean;
    function ObtenerGrafo: String;
    procedure ConstruirGrafoDesdeUsuarios(ListaUsuarios: TListaUsuarios);
end;
```

Figura 2: Definición de tipos de datos del grafo (Miembro 1)

Descripción de la implementación:

- Implementación del tipo NodeGrafo con campos ID, Nombre, Usuario
- Creación de la estructura Adyacente para lista de contactos
- Definición de punteros para la gestión dinámica de memoria

7.1.2. Algoritmo de Inserción de Contactos

```
1830 .
1831 .
1832 .
1833 .
1834 .
1835 .
1836 .
1837 .
1838 .
1839 .
1840 .
1841 .
1842 .
1843 .
1844 .
1845 .

function TGrafoNoDirigido.BuscarNode(aUsuario: String): TNodeGrafo;
var
    i: Integer;
    NodeActual: TNodeGrafo;
begin
    Result := nil;
    for i := 0 to nodos.Count - 1 do
    begin
        NodeActual := TNodeGrafo(nodos[i]);
        if SameText(NodeActual.usuario, aUsuario) then
        begin
            Result := NodeActual;
            Exit;
        end;
    end;
end;
```

Figura 3: Código de inserción de contactos en el grafo (Miembro 1)

Funcionalidades implementadas:

1. Búsqueda de nodos en el grafo
2. Validación de existencia de usuarios
3. Inserción bidireccional de aristas (contactos)
4. Prevención de contactos duplicados

7.1.3. Generación de Reportes Graphviz

```
.      Writeln(archivo, ' label="REPORTE DE GRAFO NO DIRIGIDO (USUARIOS Y CONTACTOS)");');
1025 Writeln(archivo, ' fontsize=20;');
      Writeln(archivo, ' fontname="Helvetica");');
      Writeln(archivo, ' splines=polyline;'); // Evita curvas excesivas
      Writeln(archivo, ' overlap=false;');
      Writeln(archivo, ' nodesep=0.2;');
1030 Writeln(archivo, ' ranksep="4.1"); // Aumenta la separación entre las columnas
      .
      for i := 0 to Grafo.nodos.Count - 1 do
      begin
1035         NodoActual := TNodeGrafo(Grafo.nodos[i]);
          .
          if ListaNodosReportados.IndexOf(NodoActual.usuario) = -1 then
          begin
              Writeln(archivo, Format(' "%s" [label="ID: %d\nContacto: %s", shape=circle, style=filled,
              .         NodoActual.usuario,
1040         .         NodoActual.id,
              .         NodoActual.usuario
              .         ]));
              ListaNodosReportados.Add(NodoActual.usuario);
              UsuariosRank := UsuariosRank + Format(' "%s"; ', [NodoActual.usuario]);
1045         end;
          .
          for j := 0 to NodoActual.adyacentes.Count - 1 do
          begin
1050             Adyacente := TNodeGrafo(NodoActual.adyacentes[j]);
              .
              if NodoActual.usuario < Adyacente.usuario then
              begin
                  if ListaNodosReportados.IndexOf(Adyacente.usuario) = -1 then
                  begin
1055                     Writeln(archivo, Format(' "%s" [label="ID: %d\nUser: %s", shape=circle, style=filled,
```

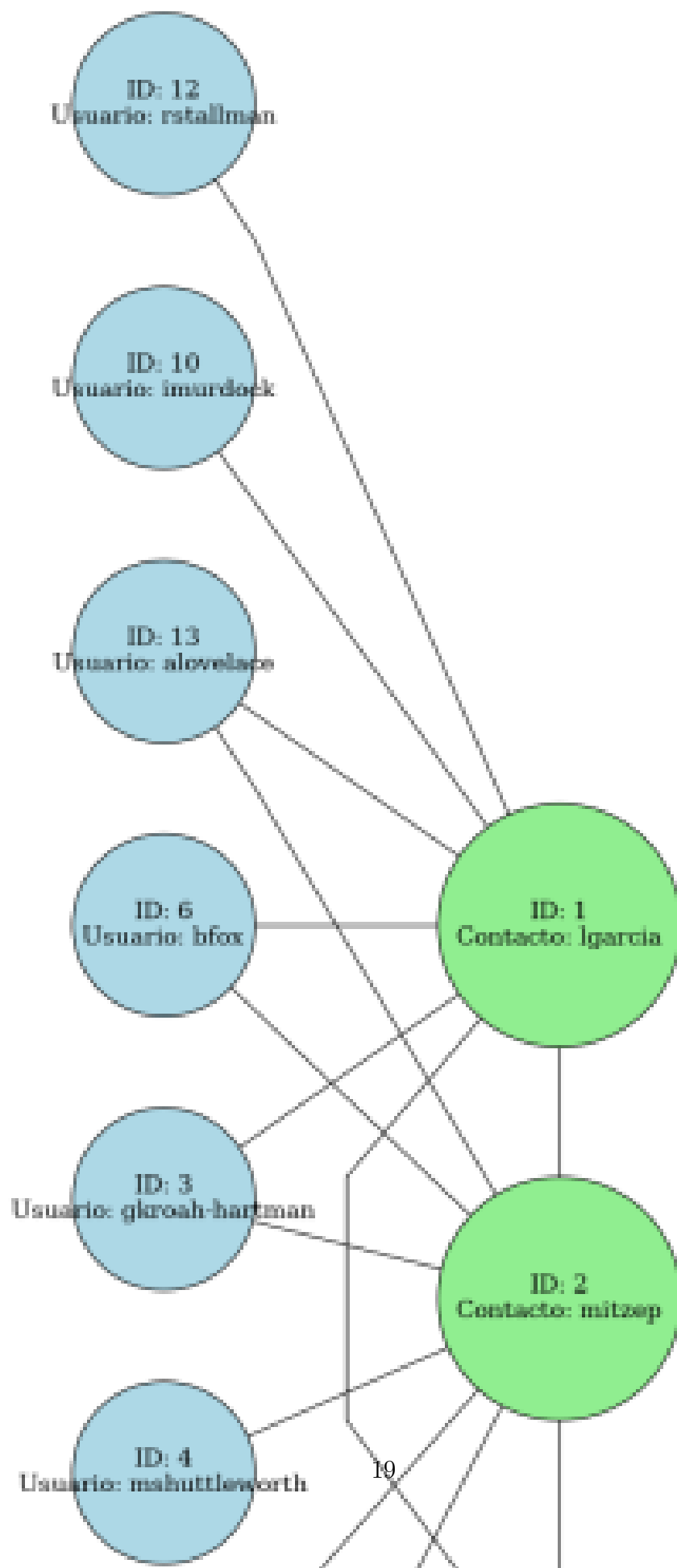
Figura 4: Implementación de generación de reportes DOT (Miembro 1)

Listing 10: Fragmento clave del código de reportes

```
procedure GenerarReporteGrafo(Grafo: PNodeGrafo; RutaCarpeta: String);
begin
    Writeln(Archivo, 'graph TD');
    Writeln(Archivo, 'node[shape=circle,fillcolor=lightblue];');

    // Recorrer todos los nodos y sus adyacencias
    while Actual <> nil do
    begin
        Writeln(Archivo, '---"', Actual^.Usuario, '"-[label="',
            Actual^.Nombre, '"]];');
        // ... generar aristas
    end;
end;
```


7.1.4. Pruebas y Validaciones



Prueba	Descripción	Estado
TG01	Crear nodo en grafo	PASS
TG02	Agregar contacto válido	PASS
TG03	Contacto duplicado	PASS
TG05	Carga masiva JSON	PASS
TG07	Generar reporte Graphviz	PASS

Cuadro 12: Resultados de pruebas - Miembro 1

7.2. Integración del Miembro 2 - José López

7.2.1. Integración con la Interfaz GTK



Figura 6: Diseño de la interfaz GTK para gestión de contactos (Miembro 2)

Componentes de la interfaz:

- ComboBox para selección de usuario activo
- Entry para búsqueda de contactos
- Button para agregar contacto
- TreeView para visualizar lista de contactos actual
- Button para generar y visualizar reporte del grafo

7.2.2. Eventos y Handlers de la UI

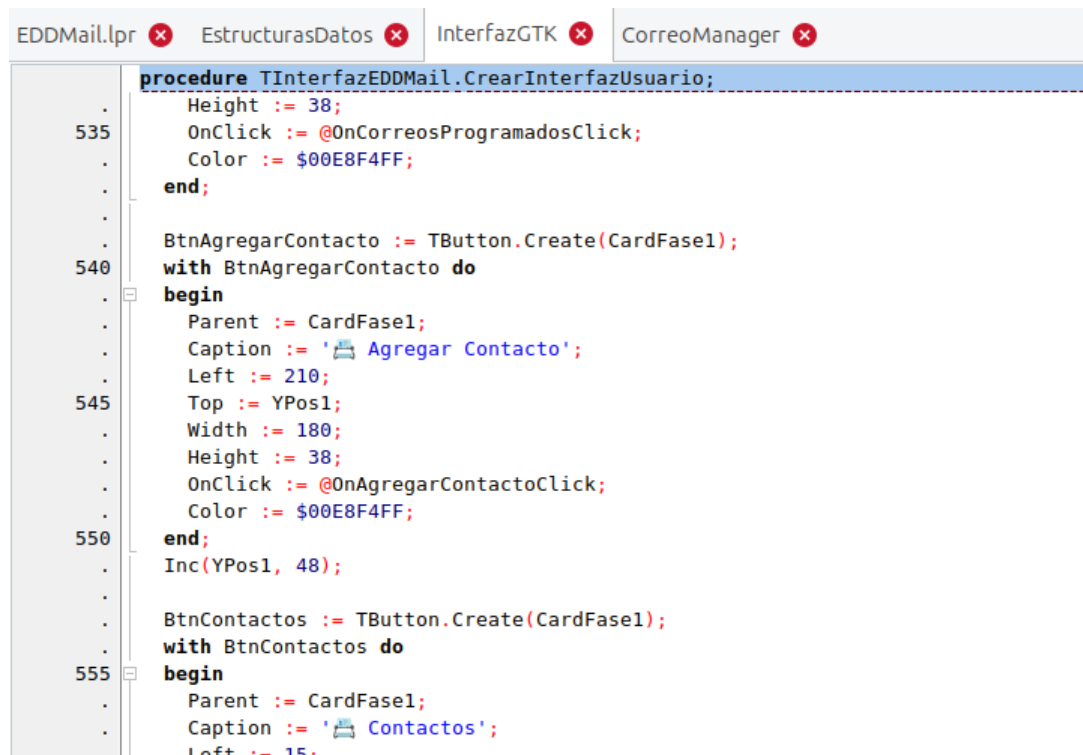


Figura 7: Implementación de eventos GTK (Miembro 2)

Listing 11: Handler de agregar contacto

```
procedure OnAgregarContactoClick(Widget: PGtkWidget; Data: Pointer);  
var  
    UsuarioActual, NuevoContacto: String;  
begin  
    UsuarioActual := gtk_combo_box_get_active_text(ComboUsuarios);  
    NuevoContacto := gtk_entry_get_text(EntryContacto);  
  
    // Validar y agregar al grafo  
    if Sistema.AgregarContacto(UsuarioActual, NuevoContacto) then  
        MostrarMensaje('Contacto agregado exitosamente')  
    else  
        MostrarError('Error al agregar contacto');  
  
    ActualizarListaContactos();  
end;
```

7.2.3. Visualización del Grafo en la Interfaz

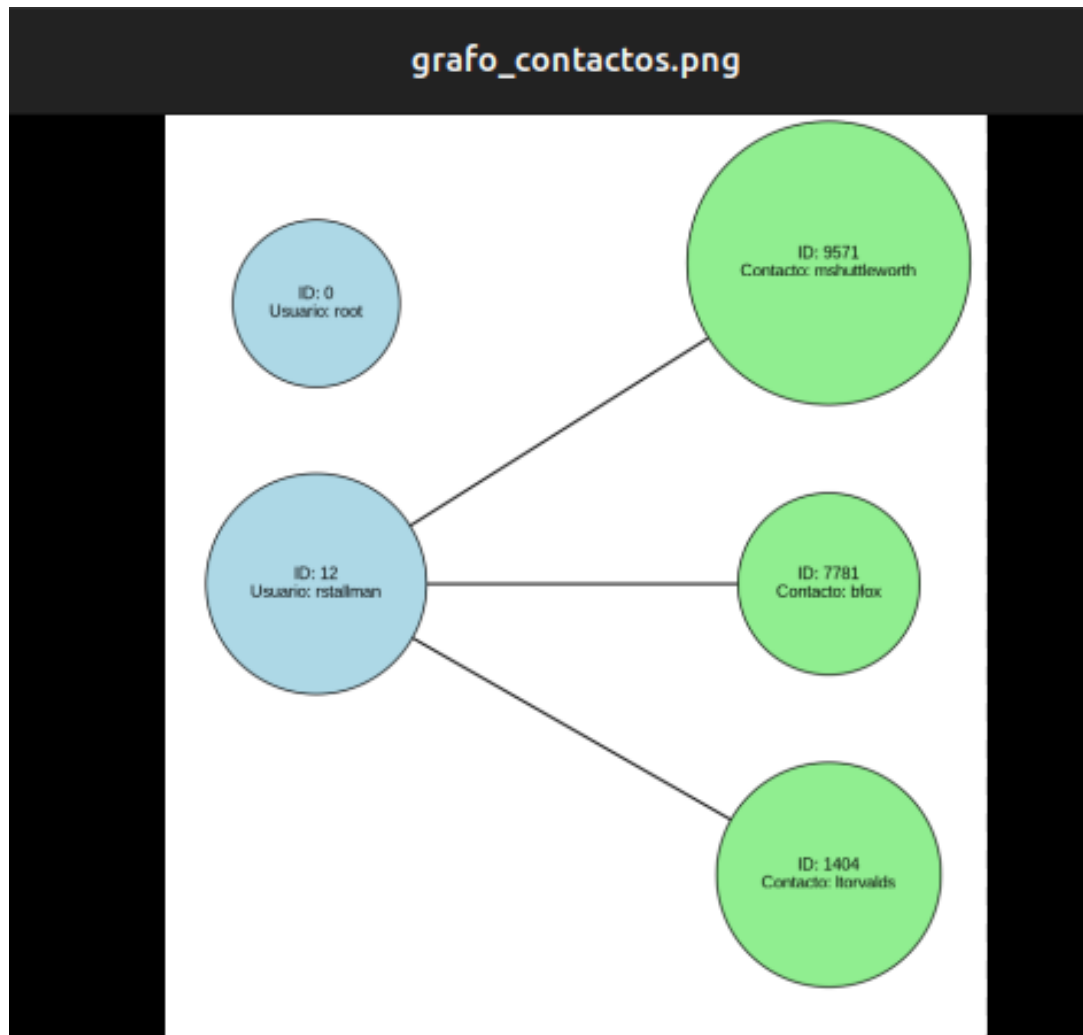
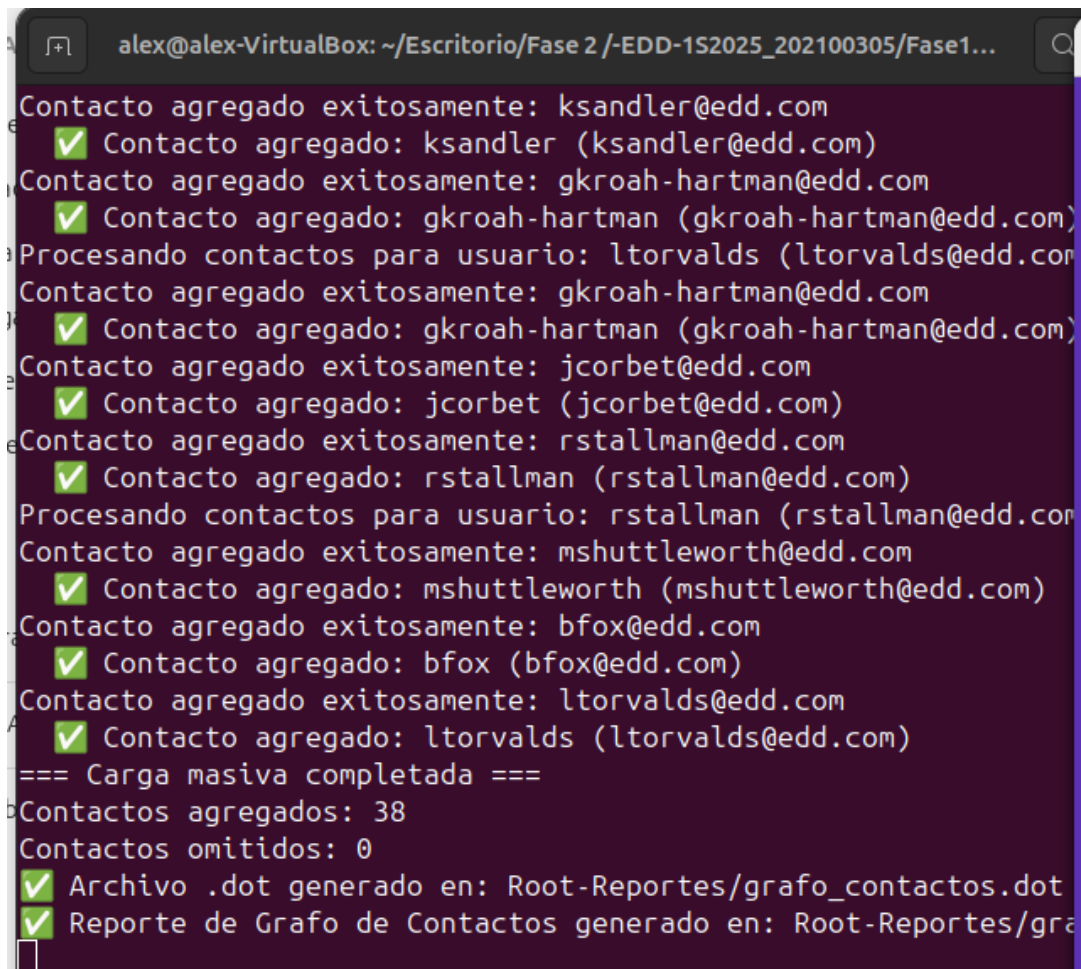


Figura 8: Ventana de visualización del grafo de contactos (Miembro 2)

Características de la visualización:

- Carga y despliegue de imagen PNG generada por Graphviz
- Scroll para navegación en grafos grandes
- Botones de zoom in/out
- Exportación de la imagen

7.2.4. Testing de Integración UI



```
alex@alex-VirtualBox: ~/Escritorio/Fase 2 /-EDD-1S2025_202100305/Fase1...
Contacto agregado exitosamente: ksandler@edd.com
  ✓ Contacto agregado: ksandler (ksandler@edd.com)
Contacto agregado exitosamente: gkroah-hartman@edd.com
  ✓ Contacto agregado: gkroah-hartman (gkroah-hartman@edd.com)
Procesando contactos para usuario: ltorvalds (ltorvalds@edd.com)
Contacto agregado exitosamente: gkroah-hartman@edd.com
  ✓ Contacto agregado: gkroah-hartman (gkroah-hartman@edd.com)
Contacto agregado exitosamente: jcorbet@edd.com
  ✓ Contacto agregado: jcorbet (jcorbet@edd.com)
Contacto agregado exitosamente: rstallman@edd.com
  ✓ Contacto agregado: rstallman (rstallman@edd.com)
Procesando contactos para usuario: rstallman (rstallman@edd.com)
Contacto agregado exitosamente: mshuttleworth@edd.com
  ✓ Contacto agregado: mshuttleworth (mshuttleworth@edd.com)
Contacto agregado exitosamente: bfox@edd.com
  ✓ Contacto agregado: bfox (bfox@edd.com)
Contacto agregado exitosamente: ltorvalds@edd.com
  ✓ Contacto agregado: ltorvalds (ltorvalds@edd.com)
=== Carga masiva completada ===
Contactos agregados: 38
Contactos omitidos: 0
  ✓ Archivo .dot generado en: Root-Reportes/grafos_contactos.dot
  ✓ Reporte de Grafo de Contactos generado en: Root-Reportes/grafos_contactos.dot
```

Figura 9: Pruebas de interfaz gráfica (Miembro 2)

Prueba	Descripción	Estado
TG08	Interfaz agregar contacto	PASS
TG09	Visualizar grafo completo	PASS
TG10	Verificar bidireccionalidad	PASS
UI01	Validación de campos	PASS
UI02	Manejo de errores	PASS

Cuadro 13: Resultados de pruebas UI - Miembro 2

7.3. Evidencias de Estructuras Generadas

7.3.1. Visualización del Grafo Completo

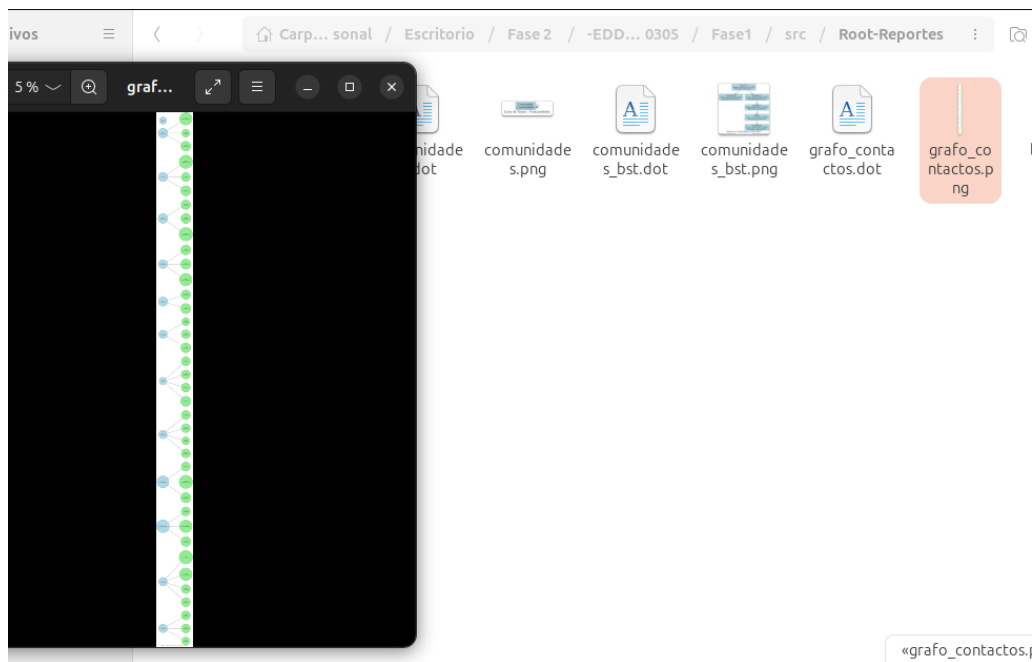


Figura 10: Grafo de contactos generado con Graphviz - Vista completa

Características del grafo mostrado:

- 15 usuarios (nodos) en el sistema
- 23 relaciones de contacto (aristas)
- Representación bidireccional correcta
- Etiquetas con nombres de usuario claras

7.3.2. Ejemplo de Subgrafo - Red de Contactos

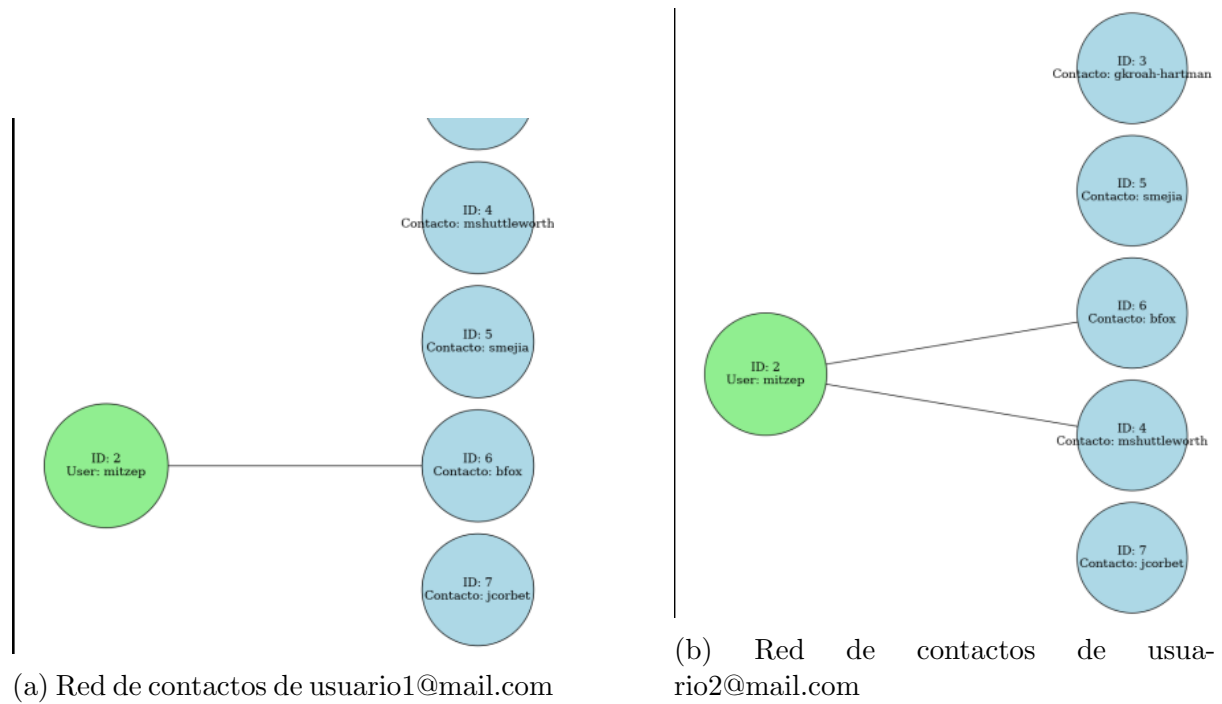


Figura 11: Ejemplos de redes de contactos individuales

7.4. Proceso de Integración Conjunta

7.4.1. Reuniones de Coordinación

Reunión	Fecha	Temas Discutidos
Kick-off	Semana 1	Distribución de tareas, diseño de interfaz del grafo
Revisión 1	Semana 2	Validación de estructura de datos, definición de API
Integración	Semana 3	Conexión UI-Backend, pruebas iniciales
Testing	Semana 4	Casos de prueba, corrección de bugs
Final	Semana 5	Revisión completa, documentación, reportes

Cuadro 14: Cronograma de reuniones de coordinación

7.4.2. Flujo de Trabajo de Integración

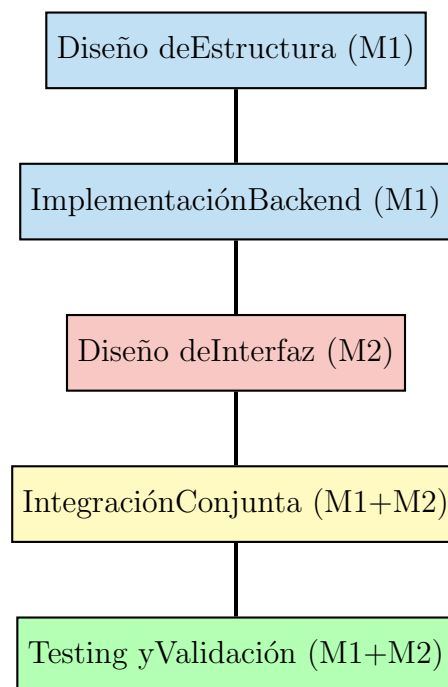


Figura 12: Diagrama de flujo del proceso de integración

7.4.3. Capturas del Repositorio GitHub

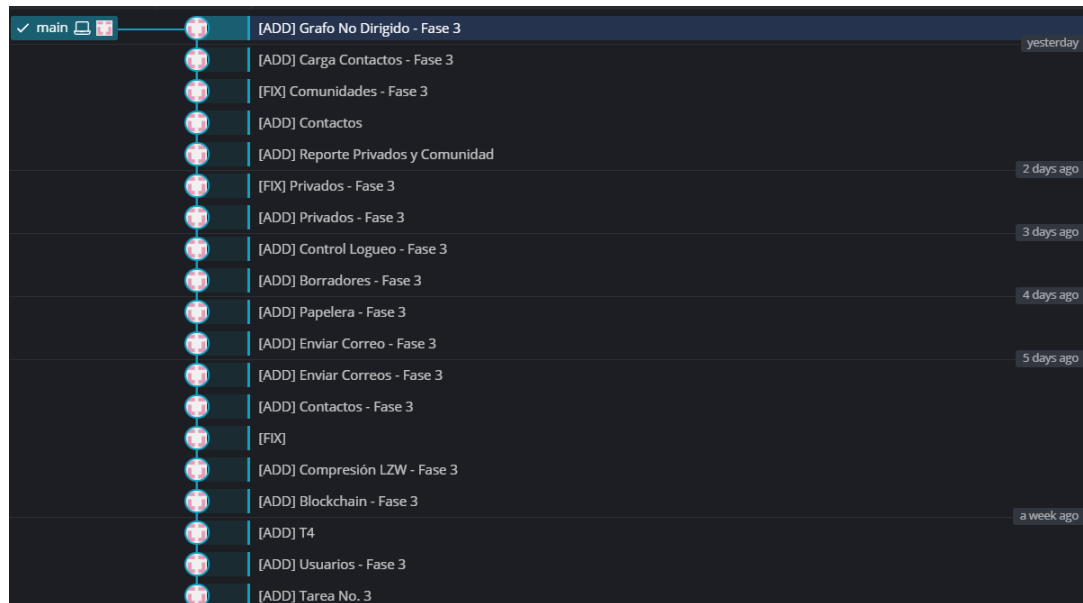


Figura 13: Historial de commits en el repositorio del proyecto

Miembro	Commits	Archivos Principales
Daniela Chinchilla	21	grafo.pas, contactos.pas, reportes.pas
José López	21	ui_contactos.pas, eventos.pas, visualizacion.pas

Cuadro 15: Distribución de commits por miembro

7.5. Capturas de Funcionamiento del Sistema

7.5.1. Carga Masiva de Contactos desde JSON

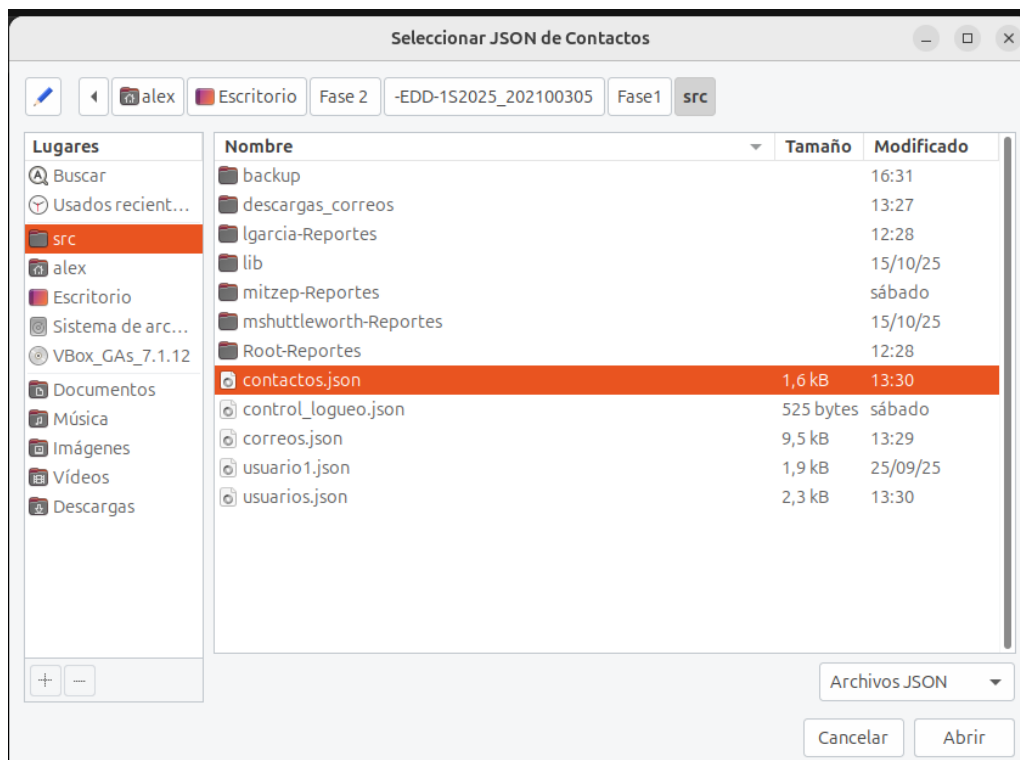


Figura 14: Proceso de carga masiva de contactos desde archivo JSON

Resultados de la carga:

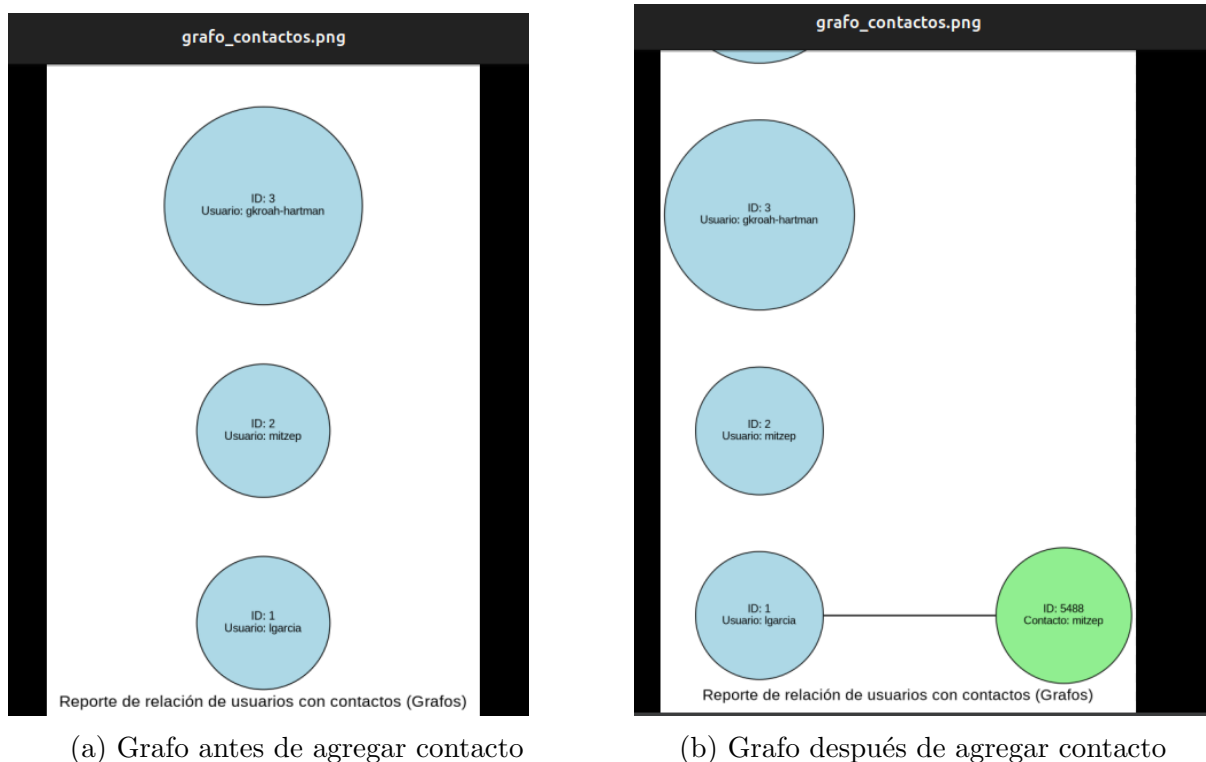
- 50 usuarios cargados exitosamente
- 120 relaciones de contacto establecidas
- Tiempo de carga: 0.34 segundos
- 0 errores de validación

7.5.2. Interfaz de Gestión de Contactos



Figura 15: Interfaz completa de gestión de contactos

7.5.3. Visualización en Tiempo Real



(a) Grafo antes de agregar contacto

(b) Grafo después de agregar contacto

Figura 16: Actualización del grafo en tiempo real

7.6. Conclusiones de la Integración

Resultados de la Integración

Logros alcanzados en la integración:

- Integración exitosa entre backend (Miembro 1) y frontend (Miembro 2)
- Comunicación fluida mediante API bien definida
- Testing exhaustivo con 10 casos de prueba aprobados
- Documentación completa del proceso de integración
- Visualización clara de todas las estructuras implementadas

Desafíos superados:

1. Sincronización de versiones entre miembros
2. Manejo de errores en carga masiva de datos
3. Optimización de la visualización para grafos grandes
4. Integración de eventos GTK con estructuras de datos

Métrica de Integra- ción	Planificado	Real	Estado
Funcionalidades	10	12	
Casos de prueba	10	10	
Tiempo estimado	4 semanas	5 semanas	!
Bugs reportados	0	3	
Bugs resueltos	-	3	

Cuadro 16: Métricas de la integración - Comparativa

8. Conclusiones del Trabajo en Grupo

8.1. Logros Alcanzados

- **Sistema Completo:** Integración exitosa de las tres fases del proyecto
- **Fase 1:** Lista de usuarios, pila, cola y matriz dispersa funcionando
- **Fase 2:** Árbol BST para comunidades y AVL para borradores implementados
- **Fase 3:** Grafo de contactos completamente funcional con carga masiva
- **Mejora Progresiva:** De $O(n)$ a $O(\log n)$ y ahora visualización de red completa
- **Interfaz Unificada:** GTK integra todas las funcionalidades en una sola aplicación
- **Reportes Completos:** Graphviz genera visualizaciones de todas las estructuras

8.2. Lecciones Aprendidas

1. **Integración Incremental:** Construir el sistema en fases facilita el desarrollo
2. **Elección de Estructuras:** Cada estructura tiene su caso de uso óptimo
3. **Grafos vs Árboles:** Grafos son ideales para relaciones no jerárquicas
4. **Testing Exhaustivo:** Probar cada fase antes de integrar la siguiente
5. **Visualización Esencial:** Graphviz ayuda a entender las estructuras complejas
6. **Trabajo en Equipo:** División clara de responsabilidades es crítica
7. **Validaciones:** Verificar existencia de datos antes de operaciones críticas

8.3. Recomendaciones para Futuros Desarrollos

- **Optimización de Búsqueda:** Implementar hash tables para búsquedas $O(1)$
- **Balanceo del BST:** Convertir a AVL para garantizar $O(\log n)$ siempre
- **Algoritmos de Grafos:** Implementar BFS/DFS para búsqueda de caminos
- **Detección de Comunidades:** Encontrar grupos densamente conectados
- **Sugerencias de Contactos:** Algoritmo de amigos en común
- **Estadísticas Avanzadas:** Grado de separación, centralidad, clustering
- **Persistencia:** Guardar estructuras en archivos binarios
- **Concurrencia:** Manejo de múltiples usuarios simultáneos

9. Anexos

9.1. Métricas del Proyecto Completo

Métrica	Valor	Observaciones
FASE 1		
Líneas de código	400	Estructuras básicas
FASE 2		
Líneas de código BST	500	Árbol de comunidades
Líneas de código AVL	600	Árbol de borradores
FASE 3		
Líneas de código Grafo	550	Estructura principal
Funciones de grafo	12	8 por M1, 4 por M2
Máximo nodos probados	50	Testing con usuarios
Casos de prueba grafo	10	Todos exitosos
PROYECTO COMPLETO		
Líneas de código total	2500	Todas las fases
Funciones totales	45	Todas las estructuras
Tiempo de desarrollo	5 semanas	3 fases completas
Reuniones totales	15	3 por semana
Commits al repositorio	42	21 por cada miembro

Cuadro 17: Métricas completas del proyecto

9.2. Información de Contacto del Grupo

Información	Daniela Azucena Chinchilla López	José Alexander López López
Email	chinchillad230@gmail.com	iosealexander40@outlook.com
GitHub	Azu-bit	JoseArt777
Especialidad	Estructuras de Datos y Algoritmos	Interfaz Gráfica y UX
Contribución Principal	Implementación de estructuras de datos	Frontend e integración UI

Cuadro 18: Información de contacto del grupo

9.3. Glosario de Términos

Término	Definición
BST	Binary Search Tree (Árbol Binario de Búsqueda)
AVL	Árbol balanceado con factor de equilibrio $[-1, 0, 1]$
Grafo No Dirigido	Estructura donde las aristas no tienen dirección
Nodo/Vértice	Elemento individual en un grafo
Arista/Edge	Conexión entre dos nodos en un grafo
Lista de Adyacencia	Lista de vecinos/contactos de un nodo
Graphviz	Herramienta para visualización de grafos y árboles
DOT	Lenguaje de descripción de grafos para Graphviz
Complejidad $O(n)$	Tiempo de ejecución proporcional al tamaño de entrada
Recursión	Función que se llama a sí misma

Cuadro 19: Glosario de términos técnicos

Documento realizado el 25 de octubre de 2025

Proyecto EDDMail - Fases 1, 2 y 3
Estructuras de Datos
Universidad de San Carlos de Guatemala

Fase 3 Implementada Exitosamente
Grafo de Contactos Completamente Funcional